10. Caching: The Secret Behind It All.

What is Caching :-

- **Caching** is the process of making faster data accessing over the internet by storing it in **RAM** with tools like **Redis**, the data that is stored in the RAM is called **cache**.
  - It reduces time to retrive data by keeping the **subset** (chunk) of most frequently accessed data in the cache.
  - **Example** -> If many people search weather information many times the google will show the weather from in-memory so that it can be fetched fast. **Remember** it is stored in subset of the information.

Why Caching is Used :-

- **Reduce Latency** -> Fast data retrival operations.
- **Reduce Database Load** -> Redis can provide the information if it is cached and reduce the by not accessing evey single information from the database.
- **Reduce Computation** -> Reuse result of expensive operation.
- **Improve Scalability** -> Serve more request, without hitting the slow database everytime for fetcing data.

Cache Hit vs. Cache Miss :-

- **Cache Hit** -> When the subset of most frequently data is found in the in-memory(RAM) storage then the data can be served by the Redis directly, it is said as cache hit.
- **Cache Miss** -> When the subset of most frequently data is not found in the Redis then it is retrived from the database and then stored in the RAM.

Cache Eviction :-

- Eviction removes cached ntries to free the capacity for new data. The RAM is limited so we categorize the data in 2 parts depending how frequently they are accessed so some cached entries can be removed. The two types are :-

1. **LFU (Least Frequently Used) ->** Remove the items which have been accessed from the lowest frequency

overall.

2. **LRU (Least Recently Used) ->** Remove the items that have not be recently used or have not been accessed from the longer period of time.

- **No Eviction ->** A cache configuration where the data is not automatically removed for the newer data writes when the cache reaches its maximum memory limit.
- **TTL (Time To Live) ->** It is a expiration mechanism, It decides for how much the data will be cached and remain valid for retrival from the server. Removes the data if it is above the pre-set amount of time.

What is CDN (Content Delivery Network) :-

- A geographical distributed network of server that stores the copy of the data worldwide. Its main idea is to bring server close to the user when bandwith is limiting its performance and distribute the workload throught the server.
- **Origin server** is the main server which have the orignal data and the **Edge server** have the copy of that data and is set up to provide good bandwith and **PoP (Point of Presence)** is the CDN infrastructure that is deployed.

Levels of caching :-

Level	Examples	Purpose

Network	CDN, DNS cache	Reduce latency and repeatedly lookups. **CDN cache** is when user request anything from the main server and its static copy gets saved in the edge server reducing latency. AND **DNS cache** also works kinda same like it stores previously fetched Domain-to-IP lookups so that the client can get the respone faster.
Hardware	CPU L1, L2, L3	**RAM** is fast but not as faster as these L1, L2, L3 because they are closer to the cpu and if cpu fetches every data from ram then the cpu will spend most of its time sitting idle. So these chips have **low capacity but transfers data at very high speed.**

Software	Redis and Memcached	This is the primary layer that was designed for the backend engineers to protect the database and accelerates API responses. **Redis** is in-memory storage used for fast read operations, **Memcached** is also in-memory storage in which we can store data in key-value pair and can speed up dynamic applications.

Caching Strategies :-

- **Cache-Aside (Lazy Caching) ->** The application check the cache, on Cache Hit is will return the cached value. On Cache Miss it will retrive the data from the database and save it in cache and then serve the data to the user.
  - **Write-Through Caching ->** It is a caching strategy in which the data is written on the both cache and in database at the same time. It also adds a overhead that it requires strict faliure handling. **Flow ->** Data goes to cache and database together -> The write is only considered complete when the data is saved in both of the storage -> This create an active sync between cache and database and they match the data with each other.

Network-Level Caching :-

CDN (Content Delivery Network) ->

- Caches the content of the globally distributes edge servers so user can retrive the data from the nearby location.
- **PoP(Point of Presence):** A location where the CDN infrastructure is located.

- **Cache Hit:** Data is found in the cache of the edge server ;
  Cache Miss: Data will be retrieved from the main server and be saved in the edge server cache.
- **TTL:** Decides for how much time the data will be retained before expiart or revocation.

DNS Caching ->

- It Stores **Domain-to-IP lookups** in the cache for the reduction in latency and expensive computation.
  - **Example ->** (api.example.com → 93.184.216.34)

Cache Invalidation :-

- When the data saved in the db gets updated or deleted so this behavior must be applied on the cache on the Redis/ Ram.

Backend Use Cases :-

- Avoid repeated executing expensive queries.
- Increases the read speed of the data.
- Reduces repeated requests to external service.
- Sets some rate limiting for how many time the data can be retrived in the particular period of time.
- Store session ID, auth tokens, login session and user state in Redis so every API doesn't hit database everytime.
- It can be also used to prevent rate-limit penalties.